Intune Win32 Application Packaging & Deployment

Implementation Solution Guide

Version 1.0.0 · 2026-06-28

Contents

Executive Summary — Intune Win32 Application Packaging & Deployment (Microsoft Intune) 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Intune Win32 Application Packaging & Deployment 5

What the Win32 app model is . . . 5

How it works . . . 5

Detection vs. requirement rules (the distinction that matters) . . . 5

What you can control . . . 5

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — Intune Win32 Application Packaging & Deployment 6

2.1 Licensing . . . 7

2.2 Device enrollment and the Intune Management Extension . . . 7

2.3 The Microsoft Win32 Content Prep Tool . . . 7

2.4 A clean packaging source folder . . . 7

2.5 Known, tested install and uninstall command lines . . . 7

2.6 A reliable detection method . . . 8

2.7 Requirement rule values . . . 8

2.8 Dependencies identified (if any) . . . 8

2.9 Source / distribution hygiene . . . 8

Checklist . . . 8

Global Requirements — Intune Solutions (category-level) 9

G1. Microsoft Entra ID tenant . . . 9

G2. Microsoft Intune subscription . . . 9

G3. Administrative roles . . . 9

G4. Device enrollment in Intune . . . 9

G5. Pilot security group . . . 9

G6. Supported Windows baseline . . . 9

G7. Network access to Microsoft cloud endpoints . . . 10

3. Implementation Guide — Intune Win32 Application Packaging & Deployment 10

Named objects (reuse these names verbatim everywhere) . . . 10

Step 1 — Wrap the source into a `.intunewin` . . . 10

Step 2 — Create the ring groups . . . 11
Step 3 — (If needed) Package and create the dependency app first . . . 11
Step 4 — Create the main Win32 app . . . 11
Step 5 — Set install/uninstall commands and behavior . . . 11
Step 6 — Author requirement rules . . . 11
Step 7 — Author detection rules . . . 12
Step 8 — Declare dependencies and supersedence . . . 12
Step 9 — Assign to the pilot ring . . . 12
Step 10 — Apply and validate on a pilot device . . . 12
Step 11 — Expand to the broad ring . . . 13
5. Testing Guide — Intune Win32 Application Packaging & Deployment 13
Test 1 — Package built correctly . . . 13
Test 2 — App reports applicable, not blocked by requirements . . . 13
Test 3 — Install succeeds on the device . . . 13
Test 4 — Detection rule is correct (reporting is true) . . . 13
Test 5 — Dependency installed first . . . 14
Test 6 — Supersedence replaces the old version . . . 14
Test 7 — Uninstall works . . . 14
Test 8 — Fleet report . . . 14
Pass/fail gate . . . 14
6. Troubleshooting Guide — Intune Win32 Application Packaging & Deployment 14
A. App shows Failed (but seems installed) or loops reinstalling . . . 14
B. App shows Not applicable . . . 15
C. Install fails outright . . . 15
D. Dependencies install out of order . . . 15
E. Supersedence didn't replace the old version . . . 15
F. IME not present / nothing installs . . . 16
G. Diagnostics quick reference . . . 16
H. When to escalate . . . 16
7. Best Practices — Intune Win32 Application Packaging & Deployment 16
8. Deliverables Summary — Intune Win32 Application Packaging & Deployment 17
Documentation . . . 17
PowerShell scripts . . . 17
What the buyer receives . . . 18
Outcome . . . 18
Appendix A — PowerShell Scripts 19

Executive Summary — Intune Win32 Application Packaging & Deployment (Microsoft Intune)

Workplace Marketplace · Intune Solutions

Getting a Windows application onto your fleet shouldn't be a coin flip. Done by hand, app deployments fail silently: the installer runs but Intune reports the wrong state, detection rules are guesses, dependencies install out of order, and a bad package lands on everyone at once. **Win32 app deployment** through Microsoft Intune fixes this — you build a `.intunewin` package, set exact install and uninstall commands, author detection and requirement rules that are actually correct, model dependencies and supersedence, and assign in **rings** so a bad build never reaches production.

This is a complete, **deploy-it-today** package for packaging and deploying Windows applications with Microsoft Intune, written for an administrator with no prior packaging experience.

What you get

• A plain-language **Solution Overview** of the Win32 model, the Intune Management Extension, detection vs. requirement rules, dependencies, supersedence, and ring assignment.

• A **Requirements Guide** with verify / implement-if-missing for every prerequisite.

• An **Implementation Guide** of precise, click-by-click Intune steps in the required order, built around a worked example app.

• Five **PowerShell scripts** for packaging, install/detection validation, fleet reporting, safe remediation, and ring-group targeting.

• **Testing** and **Troubleshooting** guides for confident rollout.

What this delivers

A repeatable Win32 packaging method — a clean `.intunewin`, correct **install/uninstall** command lines, **detection** and **requirement** rules that report the true state, modelled **dependencies** and **supersedence**, and a **pilot / broad ring** assignment so every package is proven before it scales.

The deployment model

Assess -> Configure -> Pilot -> Validate -> Expand. The pilot ring is the gate.

Outcome

Windows applications that deploy predictably from Intune — correct detection, ordered dependencies, supersedence handled, and ring assignment so a bad package never reaches production — deployable by a single administrator and validated before fleet rollout.

1. Solution Overview — Intune Win32 Application Packaging & Deployment

What the Win32 app model is

A Win32 app in Microsoft Intune is a Windows application packaged into a single `.intunewin` file and deployed to enrolled devices by the **Intune Management Extension (IME)**. Unlike the older line-of-business (LOB) MSI installer type, the Win32 model wraps **any** installer — MSI, EXE, or a script — and gives you full control over the **install command**, the **uninstall command**, how Intune **detects** whether the app is already present, the device **requirements** the app needs, its **dependencies**, and what it **supersedes**. It is the standard, modern way to ship Windows software through Intune.

How it works

1. You wrap your installer and its source files into a `.intunewin` with the Microsoft **Win32 Content Prep Tool** (`IntuneWinAppUtil.exe`). This is a one-time packaging step per app version.
1. You create a **Windows app (Win32)** in Intune, upload the `.intunewin`, and set the exact **install** and **uninstall** command lines, the **install behavior** (system or user), and the device **restart** behavior.
1. You author a **detection rule** — the test the IME runs to decide if the app is installed (an MSI product code, a file/version, or a registry value). Detection is what makes Intune report the correct state; a wrong rule is the number-one cause of "installed but shows failed" or endless reinstall loops.
1. You author **requirement rules** — conditions a device must meet before the app is even attempted (OS architecture, minimum OS version, disk space, a registry/file/script check).
1. You optionally declare **dependencies** (apps that must install first, e.g. a runtime) and **supersedence** (a new version that replaces — and optionally uninstalls — an older app).
1. You **assign** the app to groups. The IME downloads the content, runs the install command in the chosen context, then runs the detection rule and reports **Installed / Failed / Not applicable** back to Intune.

Detection vs. requirement rules (the distinction that matters)

- A **requirement rule** answers "*should* this device get the app?" It gates applicability. If a device fails a requirement, Intune reports **Not applicable** and never installs.
- A **detection rule** answers "*is* the app already installed?" It is evaluated before install (to skip an already-present app) and after install (to confirm success). Detection drives reporting.

What you can control

- **Install / uninstall command lines** and the **install context** (system vs. user).

• **Detection** by MSI product code, file existence/version/date, registry value, or a custom detection script.

• **Requirements:** OS architecture (x64), minimum OS build, free disk space, and custom file/registry/script requirement rules.

• **Dependencies** (install order, auto-install) and **supersedence** (replace/uninstall prior).

• **Assignment** as **Required**, **Available** (Company Portal), or **Uninstall**, targeted by ring.

• **Return codes** (which exit codes mean success, soft reboot, hard reboot, or retry).

Benefits

• **Reliability** — correct detection means Intune reports the true state, not a guess.

• **Any installer** — MSI, EXE, or script all ship through one consistent model.

• **Ordered installs** — dependencies guarantee a runtime lands before the app that needs it.

• **Clean upgrades** — supersedence replaces old versions without leaving duplicates.

• **Safe rollout** — ring assignment means a bad package hits a pilot, never the whole fleet at once.

Limitations (be honest with buyers)

• A `.intunewin` has a content size ceiling (multi-GB); very large suites may need a different approach.

• Detection and requirement rules are only as good as you author them — guesses cause false states.

• The IME must be present and healthy on the device; it installs automatically on enrolled Windows.

• Per-user installs report per logged-on user; most line-of-business apps install in **system** context.

• Supersedence is one level deep per chain step; deep upgrade chains must be modelled deliberately.

The standard deployment model

> ***Wrap the source into a*** `.intunewin` ***→ create the Win32 app with correct install/uninstall commands → author detection and requirement rules → model dependencies and supersedence → assign to a pilot ring → validate → expand to broad.*** *This is the repeatable way:* ***Assess → Configure → Pilot → Validate → Expand.*** *A bad build never reaches production because the pilot ring is the gate.*

2. Requirements Guide — Intune Win32 Application Packaging & Deployment

Verify **every** requirement before packaging or deploying. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

Throughout this solution we use a **worked example app** so every rule is concrete: a generic line-of-business installer named `Contoso LOB Suite` shipped as `ContosoLOB.msi` (with placeholder

source path `C:\Packaging\ContosoLOB\`). Replace the names and paths with your own app.

2.1 Licensing

Description: Intune (Plan 1). Win32 app deployment is a built-in Intune capability. **Why:** Intune stores the `.intunewin`, runs the IME, and reports app install state. **Verify:** M365 admin center -> **Billing -> Licenses**. **Implement:** Assign M365 E3/E5 or Business Premium (Intune included), or standalone Intune Plan 1.

2.2 Device enrollment and the Intune Management Extension

Description: Targeted Windows devices enrolled in Intune; the IME present and healthy. **Why:** Win32 apps are installed only by the IME on enrolled devices. **Verify:** Intune -> **Devices -> All devices** lists the device; run `WIN32-Validation.ps1` (checks for the `Microsoft Intune Management Extension` service). **Implement:** Enable automatic enrollment and enroll the device. The IME installs automatically once a Win32 app or PowerShell script is assigned to the device.

2.3 The Microsoft Win32 Content Prep Tool

Description: `IntuneWinAppUtil.exe`, Microsoft's free tool that wraps source into `.intunewin`. **Why:** Intune only accepts the `.intunewin` package format for Win32 apps. **Verify:** The tool is present on your packaging workstation (run `WIN32-New-IntuneWinPackage.ps1`, which locates or prompts for it). **Implement:** Download it from Microsoft's official repository at `https://github.com/microsoft/Microsoft-Win32-Content-Prep-Tool` (Releases). Place `IntuneWinAppUtil.exe` in a known folder such as `C:\Tools\IntuneWinAppUtil\`. Do not redistribute the tool inside this package; obtain the current version from Microsoft.

2.4 A clean packaging source folder

Description: A folder containing the installer (the **setup file**) plus everything it needs. **Why:** The Content Prep Tool captures one source folder; missing files cause install failures. **Verify:** Your source folder (e.g. `C:\Packaging\ContosoLOB\`) holds the installer and any transforms/config it references, and nothing extra. **Implement:** Stage the installer (`ContosoLOB.msi`) and dependencies into one folder; remove stray files. Note the **setup file name** — you pass it to the Content Prep Tool.

2.5 Known, tested install and uninstall command lines

Description: The exact silent install and silent uninstall commands for the app. **Why:** Intune runs these verbatim; an interactive or wrong command stalls or fails the deployment. **Why these specific examples:** MSIs use `msiexec`; EXEs use a vendor-specific silent switch. **Verify:** Run the commands manually on a clean test VM and confirm a silent, successful install and a clean uninstall, both returning exit code `0`. **Implement (worked example, MSI):**

- Install: `msiexec /i "ContosoLOB.msi" /qn /norestart`
- Uninstall: `msiexec /x {PRODUCT-GUID} /qn /norestart`

For an EXE installer, use the vendor's documented silent switch (commonly `setup.exe /silent /norestart` or `/S`); confirm it in the vendor docs.

2.6 A reliable detection method

Description: A deterministic way to confirm the app is installed (MSI product code, a file with a version, or a registry value). **Why:** Detection drives Intune's reported state; a guess causes false Installed/Failed results. **Verify:** After a manual install, capture the MSI **product code**, or the installed **file + version** (e.g. `C:\Program Files\Contoso\LOB\ContosoLOB.exe`, version `4.2.0`), or the uninstall **registry key**. **Implement:** Choose one: MSI product code (best for MSIs), file-version rule, or registry-value rule. Record the chosen value — you enter it as the detection rule and reuse it in `WIN32-Validation.ps1`.

2.7 Requirement rule values

Description: The device conditions the app needs (architecture, minimum OS build, disk space). **Why:** Requirement rules stop the app from targeting devices that cannot run it (reported **Not applicable** rather than failed). **Verify:** Confirm the app's supported architecture (typically **64-bit**) and minimum Windows version from the vendor. **Implement:** Plan an x64 architecture requirement and a minimum OS build; add a custom file/registry/script requirement only if the app has a special prerequisite.

2.8 Dependencies identified (if any)

Description: Any runtime or component the app needs *before* it installs (e.g. a C++ redistributable). **Why:** Out-of-order installs fail; Intune dependencies enforce install order. **Verify:** Confirm from the vendor whether a runtime is required and whether it is already deployed as its own Win32 app. **Implement:** Package the prerequisite as its own Win32 app first, then declare it as a dependency of the main app (see Implementation Step 7).

2.9 Source / distribution hygiene

Description: A versioned, backed-up store of each `.intunewin` and its source folder. **Why:** You must be able to rebuild or re-upload a package and trace which version is deployed. **Verify:** Each package lives in a versioned folder (e.g. `\\share\Packages\ContosoLOB\4.2.0\`). **Implement:** Adopt a `App\Version\` folder convention; keep the source folder alongside the `.intunewin` so you can rebuild it.

Checklist

- Intune licensing assigned
- Devices enrolled; IME healthy — `WIN32-Validation.ps1`
- Win32 Content Prep Tool downloaded from Microsoft
- Clean packaging source folder staged
- Tested silent install + uninstall command lines (exit `0`)
- Reliable detection value captured (product code / file+version / registry)
- Requirement rule values planned (x64, min OS build)
- Dependencies identified and packaged (if any)

- Versioned package store / source hygiene in place
- Pilot + Broad Entra ring groups created (`WIN32-Set-RingTargeting.ps1`)

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** Phoenixtekk solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every Phoenixtekk solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The Phoenixtekk deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — Intune Win32 Application Packaging & Deployment

Follow these steps **in order**. Packaging (Step 1) happens on a packaging workstation; everything else is in the **Microsoft Intune admin center** (`https://intune.microsoft.com`).

> ***Model:*** *Wrap the source, create the Win32 app with correct commands, author detection and requirement rules, model dependencies and supersedence, assign to a* ***pilot*** *ring, validate, then expand to* ***broad****. The pilot ring is the gate — a bad build never reaches production.*

Named objects (reuse these names verbatim everywhere)

We use the `<Purpose> – <Scope> – <Stage>` naming convention so every object is self-describing and searchable. For the worked example app `Contoso LOB Suite` (installer `ContosoLOB.msi`):

Object	Name to create
Pilot ring group	`App Deployment – Pilot Devices – Ring 1`
Broad ring group	`App Deployment – Broad Devices – Ring 2`
Main Win32 app	`Contoso LOB Suite – Win32 – v4.2.0`
Dependency Win32 app	`VC++ Redistributable x64 – Win32 – Dependency`
Superseded prior app	`Contoso LOB Suite – Win32 – v4.1.0`
Detection rule identifier	`Contoso LOB Suite – Detection – MSI ProductCode`
Requirement rule	`Contoso LOB Suite – Requirement – x64 MinOS22H2`

Step 1 — Wrap the source into a `.intunewin`

1. On the packaging workstation, confirm the **Win32 Content Prep Tool** is present (see Requirement 2.3). Place `IntuneWinAppUtil.exe` in `C:\Tools\IntuneWinAppUtil\`.
1. Stage the clean source folder `C:\Packaging\ContosoLOB\` with `ContosoLOB.msi` inside.

2. Run the helper: `WIN32-New-IntuneWinPackage.ps1 -SourceFolder C:\Packaging\ContosoLOB -SetupFile ContosoLOB.msi -OutputFolder C:\Packaging\Output`.

3. Confirm `C:\Packaging\Output\ContosoLOB.intunewin` is produced. This is the file you upload.

Step 2 — Create the ring groups

1. Entra -> **Groups -> New group** -> **Security** -> name `App Deployment – Pilot Devices – Ring 1`

(a few representative devices). **Create**. (Or run `WIN32-Set-RingTargeting.ps1`.)

1. Create `App Deployment – Broad Devices – Ring 2` (the rest of the fleet). **Create**.

Step 3 — (If needed) Package and create the dependency app first

If the app needs a runtime, package and create it **before** the main app so it can be referenced.

1. Wrap the runtime (Step 1) and create a Win32 app named `VC++ Redistributable x64 – Win32 – Dependency`.

2. Give it its own correct install/uninstall commands and a detection rule (e.g. its registry key).

3. You do **not** have to assign the dependency separately — it can be auto-installed via the main app's dependency declaration in Step 7.

Step 4 — Create the main Win32 app

1. Intune -> **Apps -> Windows -> Add**. For **App type** choose **Windows app (Win32)**. **Select**.

2. **App package file:** browse to `C:\Packaging\Output\ContosoLOB.intunewin`. **OK**.

3. **App information: Name** = `Contoso LOB Suite – Win32 – v4.2.0`; set **Publisher** and a short **Description**. **Next**.

Step 5 — Set install/uninstall commands and behavior

On the **Program** tab:

1. **Install command:** `msiexec /i "ContosoLOB.msi" /qn /norestart`

2. **Uninstall command:** `msiexec /x {PRODUCT-GUID} /qn /norestart`

3. **Install behavior: System** (per-device; correct for most line-of-business apps).

4. **Device restart behavior: No specific action** (the `/norestart` switch handles reboots).

5. **Return codes:** leave the defaults (`0` = success, `1707` = success, `3010` = soft reboot, `1641` = hard reboot, `1618` = retry) unless the vendor documents others. **Next**.

Step 6 — Author requirement rules

On the **Requirements** tab:

1. **Operating system architecture: 64-bit**.
2. **Minimum operating system:** select your floor (e.g. **Windows 11 22H2**).
3. **Add** any custom requirement only if needed, named `Contoso LOB Suite – Requirement – x64 MinOS22H2`

(a file, registry, or script rule). **Next**.

Step 7 — Author detection rules

On the **Detection rules** tab, choose **Manually configure detection rules** (more reliable than auto), then **Add** the rule you captured in Requirement 2.6, identified as `Contoso LOB Suite – Detection – MSI ProductCode`:

• **Rule type: MSI** -> enter the **MSI product code** `{PRODUCT-GUID}` (best for MSIs); optionally

check the product version.

• *Alternative (EXE apps)* — **Rule type: File** -> path `C:\Program Files\Contoso\LOB\`, file

`ContosoLOB.exe`, **Detection method: version**, operator **greater than or equal to**, value `4.2.0`.

• *Alternative* — **Rule type: Registry** -> the app's uninstall key + a `DisplayVersion` value check.

Use **exactly one** authoritative detection rule type per app. **Next**.

Step 8 — Declare dependencies and supersedence

1. **Dependencies** tab -> **Add** -> select `VC++ Redistributable x64 – Win32 – Dependency` ->

set **Automatically install** = **Yes** so it installs first if missing. **Next**.

1. **Supersedence** tab (when replacing an older version) -> **Add** -> select the prior app

`Contoso LOB Suite – Win32 – v4.1.0` -> choose **Uninstall previous version = Yes** (clean replace) or **No** (in-place upgrade), per the installer's behavior. **Next**.

Step 9 — Assign to the pilot ring

1. **Assignments** tab -> under **Required** -> **Add group** -> select

`App Deployment – Pilot Devices – Ring 1`. (Use **Available for enrolled devices** instead if the app should be optional via Company Portal.)

1. Do **not** assign the broad ring yet — the pilot is the gate. **Next -> Review + create -> Create**.

Step 10 — Apply and validate on a pilot device

1. On a pilot device, sync policy (**Settings -> Accounts -> Access work or school -> Info -> Sync**).
2. Run `WIN32-Validation.ps1 -DetectionType MSI -ProductCode {PRODUCT-GUID}` to confirm the install

and that the detection rule matches. Complete the **Testing Guide**.

1. Run `WIN32-Report.ps1` to confirm Intune reports **Installed** for the pilot devices.

Step 11 — Expand to the broad ring

1. Only after the pilot passes the Testing Guide gate, edit the app's **Assignments** -> under **Required** -> **Add group** -> `App Deployment – Broad Devices – Ring 2`. **Save**.
1. Re-run `WIN32-Report.ps1` over the next cycle and chase any **Failed** devices with `WIN32-Remediation.ps1` (default `-WhatIf`).

Order of operations

```
Wrap .intunewin -> Ring groups -> Dependency app -> Main app ->
Install/uninstall commands -> Requirements -> Detection -> Dependencies +
supersedence -> Assign pilot -> Validate -> Expand broad
```

5. Testing Guide — Intune Win32 Application Packaging & Deployment

Run after the app applies to the pilot ring (Implementation Step 10) and again after expanding to the broad ring. The worked example is `Contoso LOB Suite – Win32 – v4.2.0`.

Test 1 — Package built correctly

Do: Confirm `WIN32-New-IntuneWinPackage.ps1` produced `ContosoLOB.intunewin` and that the upload in Intune accepted it (no size/format error). **Expected:** A valid `.intunewin` exists and the app was created.

Test 2 — App reports applicable, not blocked by requirements

Do: Intune -> **Apps -> Windows ->** `Contoso LOB Suite – Win32 – v4.2.0` **-> Device install status**. **Expected:** Pilot devices show the app as targeted and **not Not applicable** (requirement rules pass on x64 / supported OS).

Test 3 — Install succeeds on the device

Do: On a pilot device, sync, then run `WIN32-Validation.ps1 -DetectionType MSI -ProductCode {PRODUCT-GUID}`. **Expected:** The install command ran, the app is present, and the script reports the detection rule **matches** (the app is detected as installed).

Test 4 — Detection rule is correct (reporting is true)

Do: Intune -> the app **-> Device install status**; compare to `WIN32-Validation.ps1` on-device. **Expected:** Intune reports **Installed** and the on-device detection agrees — no "installed but failed" mismatch and no reinstall loop.

Test 5 — Dependency installed first

Do: On a device that lacked the runtime, confirm `VC++ Redistributable x64 – Win32 – Dependency` installed before the main app (check its detection/registry key). **Expected:** The dependency is present; the main app installed after it, in order.

Test 6 — Supersedence replaces the old version

Do: On a device with `Contoso LOB Suite – Win32 – v4.1.0`, confirm the v4.2.0 deployment replaced it (and uninstalled the old version if that option was chosen). **Expected:** Only the targeted version remains; no duplicate entries in Programs and Features.

Test 7 — Uninstall works

Do: Temporarily set an **Uninstall** assignment to a single test device (or run the uninstall command manually on a test VM). **Expected:** The uninstall command runs cleanly (exit `0`) and detection then reports the app **not** installed.

Test 8 — Fleet report

Do: Run `WIN32-Report.ps1 -AppName "Contoso LOB Suite – Win32 – v4.2.0"`. **Expected:** A clear per-device Installed / Failed / Not applicable breakdown across the assigned rings.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-6 pass on `App Deployment – Pilot Devices – Ring 1`; install + detection agree
Broad pass	Tests 3, 4, 8 pass for `App Deployment – Broad Devices – Ring 2` after expansion

Do **not** assign the broad ring until the pilot gate passes.

6. Troubleshooting Guide — Intune Win32 Application Packaging & Deployment

Use `WIN32-Validation.ps1` for on-device install + detection state, `WIN32-Report.ps1` for fleet status, and `WIN32-Remediation.ps1` (default `-WhatIf`) for safe fixes. The Intune Management Extension log on the device is `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\IntuneManagementExtension.log`.

A. App shows Failed (but seems installed) or loops reinstalling

Symptom	Cause	Remediation
Reports Failed though the app is on the device	Detection rule does not match what the installer wrote	Fix the detection rule (correct MSI product code / file path + version / registry value); confirm with `WIN32-Validation.ps1`
App reinstalls every cycle	Detection rule never matches after install	Same as above — make detection deterministic; re-sync
Reports Installed but app is missing	Detection rule matches the wrong artifact	Tighten the detection rule to a unique value (product code or versioned file)

B. App shows Not applicable

Symptom	Cause	Remediation
Device never gets the app, shows Not applicable	A requirement rule excludes it (architecture / min OS)	Confirm the device meets the requirement; correct the requirement rule if too strict (e.g. wrong min OS build)
Targeted group is correct but still N/A	Custom requirement script returns non-matching output	Review the requirement script's expected output and exit handling

C. Install fails outright

Symptom	Cause	Remediation
Non-zero exit from the installer	Wrong/interactive command line	Re-test the silent command on a clean VM; ensure `/qn` (MSI) or the vendor silent switch; expect exit `0`
Exit `1603`	Generic MSI failure (in-use files, missing prerequisite)	Check the dependency installed first; review the MSI verbose log; confirm system context
Content download / size error	`.intunewin` too large or corrupt	Rebuild with `WIN32-New-IntuneWinPackage.ps1`; reduce source size; re-upload

D. Dependencies install out of order

• Confirm the runtime is a separate Win32 app declared as a **dependency** with **Automatically install = Yes**, not merely assigned alongside. Re-sync and re-check `WIN32-Report.ps1`.

E. Supersedence didn't replace the old version

• Confirm the new app's **Supersedence** tab references the prior app and the **Uninstall previous

version** choice is correct; allow a policy cycle. Duplicate entries usually mean supersedence was not configured (a second app was deployed instead).

F. IME not present / nothing installs

- The Intune Management Extension installs automatically once a Win32 app or PowerShell script is

assigned. Confirm the device is enrolled, sync, and check that the `Microsoft Intune Management Extension` service is running (`WIN32-Validation.ps1` flags this).

G. Diagnostics quick reference

```
Get-Service -Name 'Microsoft Intune Management Extension'   # IME present/running
Get-CimInstance Win32_Product | Where-Object Name -like '*Contoso*'   # installed MSI products (slow)
Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*' |
  Select-Object DisplayName, DisplayVersion   # registry-based detection check
```

The IME log (`IntuneManagementExtension.log`) shows the install command, exit code, and detection result per app — the first place to look.

H. When to escalate

If detection is correct, requirements pass, the command line succeeds with exit `0` on a clean VM, the IME is healthy and synced, but Intune still reports Failed after a policy cycle, collect the `IntuneManagementExtension.log` and the installer's own verbose log and open a Microsoft support case.

7. Best Practices — Intune Win32 Application Packaging & Deployment

- **Assign to a pilot ring first, always.** A small `App Deployment – Pilot Devices – Ring 1` catches

a bad package before it reaches `App Deployment – Broad Devices – Ring 2` — the single most important practice. The pilot is the gate.

- **Make detection deterministic.** Use an MSI product code, or a file path + version, or a registry

value — never a vague guess. A wrong detection rule is the number-one cause of "installed but shows failed" and reinstall loops.

- **Requirement rules gate; detection rules confirm.** Use requirements (x64, minimum OS) to keep the

app off devices that can't run it (reported Not applicable), and detection only to confirm presence.

- **Test the command lines on a clean VM first.** Confirm the silent install and uninstall both return

exit code `0` before you ever package — Intune runs them verbatim.

- **Model dependencies, don't hope.** Package a required runtime as its own Win32 app and declare it as

a dependency with auto-install, so it always lands first.

• **Use supersedence for upgrades.** Replace the prior version via supersedence rather than deploying a

second app — it prevents duplicates and orphaned old versions.

• **Name everything with** `<Purpose> – <Scope> – <Stage>`**.** Self-describing names (apps, groups, detection/requirement rules) make a growing app catalog auditable and searchable.

• **Version your packages and source.** Keep each `.intunewin` and its source folder in an `App\Version\` store so you can rebuild and trace exactly what is deployed.

• **Honor** `/norestart`**.** Let return codes (`3010` soft reboot, `1641` hard reboot) and your restart policy control reboots; avoid surprise restarts mid-day.

• **Watch the report and chase laggards.** Track Installed / Failed / Not applicable with `WIN32-Report.ps1` and remediate failures with `WIN32-Remediation.ps1` (default `-WhatIf` first).

• **Document each app's deployment record.** Record the install/uninstall commands, detection value, requirements, dependencies, and ring assignments for audits and repeatability.

8. Deliverables Summary — Intune Win32 Application Packaging & Deployment

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The Win32 / `.intunewin` model, IME, detection vs. requirement rules, dependencies, supersedence, ring assignment
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered Intune steps: wrap, create app, commands, detection, requirements, dependencies, supersedence, ring assignment
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`WIN32-New-IntuneWinPackage.ps1`	Wraps a source folder into a `.intunewin` using the Microsoft Win32 Content Prep Tool (documents where to obtain the tool)
`WIN32-Validation.ps1`	Validates the install + detection rule on a device (read-only)
`WIN32-Report.ps1`	Fleet-wide Win32 app deployment success/failure report (Microsoft Graph, read-only)
`WIN32-Remediation.ps1`	Safe remediation of failed installs; default `-WhatIf`
`WIN32-Set-RingTargeting.ps1`	Builds the pilot/broad ring groups (Microsoft Graph)

What the buyer receives

- This implementation guide as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model: **Assess -> Configure -> Pilot -> Validate -> Expand.**

Outcome

Windows applications that deploy predictably from Intune — correct detection, ordered dependencies, supersedence handled, and ring assignment so a bad package never reaches production, proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

WIN32-New-IntuneWinPackage.ps1

Purpose: Builds an Intune Win32 .intunewin package from a source folder and a setup file using the Microsoft Win32 Content Prep Tool (IntuneWinAppUtil.exe). This script does NOT embed the tool; download it from Microsoft's official repository: https://github.com/microsoft/Microsoft-Win32-Content-Prep-Tool (Releases) Place IntuneWinAppUtil.exe in a known folder (default: C:\Tools\IntuneWinAppUtil) or pass its path with -ContentPrepTool. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WIN32-New-IntuneWinPackage.ps1 ` -SourceFolder C:\Packaging\ContosoLOB -SetupFile ContosoLOB.msi ` -OutputFolder C:\Packaging\Output EXPECTED RESULTS A single .intunewin file (e.g. ContosoLOB.intunewin) in the output folder, ready to upload in Intune (Apps > Windows > Add > Windows app (Win32)). .NOTES Workplace Marketplace — Intune Solutions. Packaging helper; does not change Intune state. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$SourceFolder, [Parameter(Mandatory)] [string]$SetupFile, [Parameter(Mandatory)] [string]$OutputFolder, [string]$ContentPrepTool = 'C:\Tools\IntuneWinAppUtil\IntuneWinAppUtil.exe')

How to run: powershell -ExecutionPolicy Bypass -File .\WIN32-New-IntuneWinPackage.ps1 `-SourceFolder C:\Packaging\ContosoLOB -SetupFile ContosoLOB.msi` -OutputFolder C:\Packaging\Output EXPECTED RESULTS A single .intunewin file (e.g. ContosoLOB.intunewin) in the output folder, ready to upload in Intune (Apps > Windows > Add > Windows app (Win32)). .NOTES Workplace Marketplace — Intune Solutions. Packaging helper; does not change Intune state. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$SourceFolder, [Parameter(Mandatory)] [string]$SetupFile, [Parameter(Mandatory)] [string]$OutputFolder, [string]$ContentPrepTool = 'C:\Tools\IntuneWinAppUtil\IntuneWinAppUtil.exe')

WIN32-Remediation.ps1

Purpose: For a device where a Win32 app failed, this forces an Intune/MDM sync so the IME re-evaluates and retries the assignment, and (optionally) clears the IME's local app-grace state so the install is re-attempted on the next cycle. Defaults to -WhatIf: it reports what it WOULD do and changes nothing until you pass -WhatIf:$false. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\WIN32-Remediation.ps1 Apply (elevated): powershell -ExecutionPolicy Bypass -File .\WIN32-Remediation.ps1 -WhatIf:$false EXPECTED RESULTS In preview: a list of the sync/retry actions that would run. When applied: an MDM sync is requested so the IME retries the failed Win32 app. Re-check with WIN32-Validation.ps1 / WIN32-Report.ps1. .NOTES Workplace Marketplace — Intune Solutions. Default -WhatIf; supports ShouldProcess. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact='Medium')] param([switch]$ClearImeGraceState) # Make -WhatIf the default unless the caller explicitly turns it off. if (-not $PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = $true }

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\WIN32-Remediation.ps1 Apply (elevated): powershell -ExecutionPolicy Bypass -File .\WIN32-Remediation.ps1 -WhatIf:$false EXPECTED RESULTS In preview: a list of the sync/retry actions that would run. When applied: an MDM sync is requested so the IME retries the failed Win32 app. Re-check with WIN32-Validation.ps1 / WIN32-Report.ps1. .NOTES Workplace Marketplace — Intune

Solutions. Default -WhatIf; supports ShouldProcess. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact='Medium')] param([switch]$ClearImeGraceState) # Make -WhatIf the default unless the caller explicitly turns it off. if (-not $PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = $true }

WIN32-Report.ps1

Purpose: Read-only report of how a Win32 app (or all Win32 apps) is deploying across the fleet — Installed / Failed / Not applicable / Pending counts, optionally per device. Use it to gate the pilot before expanding and to chase failures. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WIN32-Report.ps1 powershell -ExecutionPolicy Bypass -File .\WIN32-Report.ps1 -AppName "Contoso LOB Suite – Win32 – v4.2.0" -CsvPath C:\Temp\win32.csv Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A status summary per Win32 app and, with -CsvPath, a per-device CSV (device, app, install state). .NOTES Workplace Marketplace — Intune Solutions. Read-only against Graph. #> [CmdletBinding()] param([string]$AppName, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\WIN32-Report.ps1 powershell -ExecutionPolicy Bypass -File .\WIN32-Report.ps1 -AppName "Contoso LOB Suite – Win32 – v4.2.0" -CsvPath C:\Temp\win32.csv Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A status summary per Win32 app and, with -CsvPath, a per-device CSV (device, app, install state). .NOTES Workplace Marketplace — Intune Solutions. Read-only against Graph. #> [CmdletBinding()] param([string]$AppName, [string]$CsvPath)

WIN32-Set-RingTargeting.ps1

Purpose: Creates (or reuses) the two Microsoft Entra security groups used to ring Win32 app assignments, using the – – naming convention: "App Deployment – Pilot Devices – Ring 1" (a few representative devices) "App Deployment – Broad Devices – Ring 2" (the rest of the fleet) Optionally adds device objects (by Entra deviceId) to the pilot group. Assign the Win32 app to Ring 1 first, validate, then expand to Ring 2 (see the Implementation Guide). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WIN32-Set-RingTargeting.ps1 powershell -ExecutionPolicy Bypass -File .\WIN32-Set-RingTargeting.ps1 -PilotDeviceIds "id1","id2" Requires the Microsoft Graph SDK and Group.ReadWrite.All (+ Device.Read.All to add members). EXPECTED RESULTS The two ring groups exist (created if missing) and any supplied pilot devices are added to Ring 1. The script prints each group's object id for use in the app assignment. .NOTES Workplace Marketplace — Intune Solutions. Creates Entra security groups; gated by ShouldProcess. #> [CmdletBinding(SupportsShouldProcess)] param([string]$PilotGroupName = 'App Deployment – Pilot Devices – Ring 1', [string]$BroadGroupName = 'App Deployment – Broad Devices – Ring 2', [string[]]$PilotDeviceIds)

How to run: powershell -ExecutionPolicy Bypass -File .\WIN32-Set-RingTargeting.ps1 powershell -ExecutionPolicy Bypass -File .\WIN32-Set-RingTargeting.ps1 -PilotDeviceIds "id1","id2" Requires the Microsoft Graph SDK and Group.ReadWrite.All (+ Device.Read.All to add members). EXPECTED RESULTS The two ring groups exist (created if missing) and any supplied pilot devices are added to Ring 1. The script prints each group's object id for use in the app assignment. .NOTES Workplace Marketplace — Intune Solutions. Creates Entra security groups; gated by ShouldProcess. #> [CmdletBinding(SupportsShouldProcess)] param([string]$PilotGroupName = 'App Deployment – Pilot Devices – Ring 1', [string]$BroadGroupName = 'App Deployment – Broad Devices – Ring 2', [string[]]$PilotDeviceIds)

WIN32-Validation.ps1

Purpose: Read-only check, run on a target device, that confirms (a) the Intune Management Extension is present and (b) the app's detection rule matches — i.e. the app is actually detectable the way Intune will test for it. Mirror the detection rule you configured in Intune. HOW TO RUN MSI product code: powershell -ExecutionPolicy Bypass -File .\WIN32-Validation.ps1 -DetectionType MSI -ProductCode "{PRODUCT-GUID}" File + version: powershell -ExecutionPolicy Bypass -File .\WIN32-Validation.ps1 -DetectionType File ` -FilePath "C:\Program Files\Contoso\LOB\ContosoLOB.exe" -MinVersion 4.2.0 Registry value: powershell -ExecutionPolicy Bypass -File .\WIN32-Validation.ps1 -DetectionType Registry ` -RegPath "HKLM:\SOFTWARE\Contoso\LOB" -RegName "Version" -RegValue "4.2.0" EXPECTED RESULTS IME service status, and DETECTED / NOT DETECTED for the chosen rule. No changes are made. .NOTES Workplace Marketplace — Intune Solutions. Read-only; no system changes. #> [CmdletBinding()] param([ValidateSet('MSI','File','Registry')] [string]$DetectionType = 'MSI', [string]$ProductCode, [string]$FilePath, [string]$MinVersion, [string]$RegPath, [string]$RegName, [string]$RegValue)

How to run: MSI product code: powershell -ExecutionPolicy Bypass -File .\WIN32-Validation.ps1 -DetectionType MSI -ProductCode "{PRODUCT-GUID}" File + version: powershell -ExecutionPolicy Bypass -File .\WIN32-Validation.ps1 -DetectionType File `-FilePath "C:\Program Files\Contoso\LOB\ContosoLOB.exe" -MinVersion 4.2.0 Registry value: powershell -ExecutionPolicy Bypass -File .\WIN32-Validation.ps1 -DetectionType Registry` -RegPath "HKLM:\SOFTWARE\Contoso\LOB" -RegName "Version" -RegValue "4.2.0" EXPECTED RESULTS IME service status, and DETECTED / NOT DETECTED for the chosen rule. No changes are made. .NOTES Workplace Marketplace — Intune Solutions. Read-only; no system changes. #> [CmdletBinding()] param([ValidateSet('MSI','File','Registry')] [string]$DetectionType = 'MSI', [string]$ProductCode, [string]$FilePath, [string]$MinVersion, [string]$RegPath, [string]$RegName, [string]$RegValue)